JAVASCRIPT

Uporządkowana notatka do nauki

Podstawy języka, operatory, instrukcje sterujące, tablice, teksty, funkcje i wyrażenia regularne.

Jak korzystać z notatki: Najpierw przeczytaj krótkie wyjaśnienie, a potem przeanalizuj przykład kodu. W przykładach nazwy zmiennych są po polsku, żeby łatwiej było zrozumieć ich rolę.

Plan materiału

- 1. Podstawy JavaScript i zmienne
- 2. Operatory
- 3. Instrukcje warunkowe i pętle
- 4. Tablice
- 5. Łańcuchy znaków (String)
- 6. Funkcje
- 7. Wyrażenia regularne
- 8. Najważniejsze rzeczy do zapamiętania

1. Podstawy języka JavaScript

Umieszczanie kodu JavaScript na stronie

Kod JavaScript można umieścić bezpośrednio w pliku HTML albo zapisać w osobnym pliku z rozszerzeniem .js i dołączyć go do strony.

- Kod wewnątrz HTML: umieszczamy go między znacznikami <script> i </script>. Najczęściej skrypt znajduje się pod koniec elementu body, przed znacznikiem </body>. Dzięki temu elementy strony są już wczytane, gdy skrypt zacznie działać.

```
<body>
  <h1>Moja strona</h1>
  <script>
    alert("Witaj!");
  </script>
</body>
```

- Zewnętrzny plik JS: dołączamy go za pomocą atrybutu src. Ścieżka musi prowadzić do właściwego pliku. Jedna strona może korzystać z kilku plików JavaScript.

```
<script src="js/skrypt.js"></script>
```

- Kod może też reagować na zdarzenia, np. kliknięcie przycisku. W materiale pokazano taki sposób jako zapis „inline”.

Okienka dialogowe

Funkcja	Do czego służy	Co zwraca
alert(tekst)	Wyświetla komunikat i przycisk OK.	Nie zwraca użytecznej wartości (undefined).

Funkcja	Do czego służy	Co zwraca
prompt(tekst)	Wyświetla pytanie i pole do wpisania tekstu.	Wpisany tekst jako String albo null po anulowaniu.
confirm(tekst)	Prosi o potwierdzenie: OK lub Anuluj.	Boolean: true po OK, false po Anuluj.

```
alert("Witaj!");
let imie = prompt("Jak masz na imię?");
let czyGotowy = confirm("Czy jesteś gotowy?");
```

document.write() - zapisuje tekst lub kod HTML bezpośrednio w dokumencie. W starszych przykładach służy do wyświetlania treści na stronie; obecnie zwykle stosuje się inne sposoby zmiany zawartości strony.

Podstawowe typy danych

Typ	Co przechowuje	Przykład
Number	Liczby całkowite i ułamkowe. W JavaScript część dziesiętną zapisujemy kropką.	12, 3.14, 0.135e2
String	Tekst, czyli ciąg znaków.	"Witaj", 'Ala'
Boolean	Wartość logiczną: prawda albo fałsz.	true, false

Łączenie tekstów nazywamy konkatenacją. Operator + łączy dwa ciągi znaków w jeden.

```
let imie = "Ala";
let powitanie = "Cześć " + imie; // "Cześć Ala"
```

W tekście można zapisywać znaki specjalne za pomocą sekwencji zaczynających się od ukośnika wstecznego \ :

Zapis	Znaczenie
\n	Nowy wiersz
\t	Tabulacja pozioma
\b	Backspace
\r	Powrót karetki
\f	Nowa strona / znak podziału strony
\"	Cudzysłów
\'	Apostrof
\\	Ukośnik wsteczny

Zmienne

Zmienna przechowuje wartość, do której można odwołać się pod wybraną nazwą. W notatkach pojawiają się trzy sposoby deklarowania: var, let i const.

```
var wiek = 17;
let miasto = "Łabiszyn";
const rok = 2026;
```

- var i let - służą do deklarowania zmiennych, których wartość można później zmienić.
- const - deklaruje stałą. Nie można przypisać do niej nowej wartości, ale jeśli przechowuje tablicę lub obiekt, ich zawartość może być modyfikowana.

Zasady tworzenia nazw zmiennych:

- Nazwa może zaczynać się od litery, znaku $ albo _. Nie zaczynamy jej od cyfry.
- Wielkość liter ma znaczenie: wynik, Wynik i WYNIK to trzy różne nazwy.
- Nie używamy spacji. W razie potrzeby można rozdzielać wyrazy znakiem _.
- Nie należy używać słów zarezerwowanych przez język, np. var.

```
let wynik = 10;    // poprawna nazwa
let _wynik = 10;   // poprawna nazwa
let wynik2 = 10;   // poprawna nazwa
// let 2wynik = 10; // niepoprawna nazwa
```

2. Operatory w JavaScript

Operator wykonuje działanie na jednej lub kilku wartościach. W materiale wyróżniono operatory arytmetyczne, przypisania, porównania, trójargumentowy, logiczne, łączenia tekstów, inkrementacji i dekrementacji, bitowe oraz operatory typu.

Operatory arytmetyczne i przypisania

Operator	Znaczenie / przykład
+	Dodawanie, a dla tekstów także łączenie ciągów znaków.
-	Odejmowanie.
*	Mnożenie.
/	Dzielenie.
%	Reszta z dzielenia.
=	Przypisuje wartość po prawej stronie do zmiennej po lewej.
+=	x += y oznacza x = x + y.
-=	x -= y oznacza x = x - y.
*=	x *= y oznacza x = x * y.
/=	x /= y oznacza x = x / y.
%=	x %= y oznacza x = x % y.

Operatory porównania

Operatory porównania sprawdzają relację między wartościami. Wynikiem jest zawsze wartość logiczna: true albo false.

Operator	Znaczenie
==	Równość wartości; JavaScript może wcześniej dokonać konwersji typów.
===	Ścisła równość: zgodna wartość i typ.
!=	Różność wartości; może dokonać konwersji typów.
!==	Ścisła różność: różna wartość albo różny typ.

Operator	Znaczenie
<	Mniejsze niż.
<=	Mniejsze lub równe.
>	Większe niż.
>=	Większe lub równe.

Zapamiętaj: W praktyce najczęściej używaj === i !==, ponieważ porównują bez automatycznej zmiany typu. W materiale pojawiają się też operatory: !=, <, <=, ==, ===, >= oraz >.

Operatory logiczne

Operator	Nazwa	Kiedy wynik jest true?
!	NOT - negacja	Gdy wartość po negacji jest prawdziwa. !true daje false, a !false daje true.
&&	AND - i	Gdy oba warunki są prawdziwe.
||	OR - lub	Gdy przynajmniej jeden warunek jest prawdziwy.

```
let wiek = 17;
let maZgode = true;
console.log(wiek >= 18 && maZgode); // false
console.log(wiek >= 18 || maZgode); // true
```

Operator trójargumentowy (ternary)

To krótszy zapis prostego if-else. Najpierw sprawdzany jest warunek. Jeśli jest prawdziwy, wybierana jest wartość po znaku ?, a jeśli fałszywy - wartość po dwukropku.

```
let wiek = 17;
let komunikat = (wiek >= 18) ? "Pełnoletni" : "Niepełnoletni";
```

Inkrementacja i dekrementacja

Zapis	Znaczenie
++x	Zwiększa x o 1, a następnie zwraca nową wartość.
x++	Zwraca starą wartość x, a następnie zwiększa x o 1.
--x	Zmniejsza x o 1, a następnie zwraca nową wartość.
x--	Zwraca starą wartość x, a następnie zmniejsza x o 1.

Operatory bitowe

Operatory bitowe wykonują działania na bitach liczb. W prostych przykładach 1 oznacza bit ustawiony, a 0 - bit wyłączony.

Operator	Działanie
&	AND bitowy: wynikowy bit ma wartość 1, gdy oba bity mają 1.
^	XOR: wynikowy bit ma wartość 1, gdy bity są różne.
|	OR bitowy: wynikowy bit ma wartość 1, gdy przynajmniej jeden bit ma 1.
<<	Przesuwa bity w lewo o liczbę miejsc podaną po prawej stronie.
>>	Przesuwa bity w prawo z zachowaniem bitu znaku.
>>>	Przesuwa bity w prawo, wstawiając z lewej strony zera.

Przykład przesunięcia o jedno miejsce:

```
00010100 << 1  // bity przesuwają się o 1 w lewo
00101000

00010100 >> 1  // bity przesuwają się o 1 w prawo
00001010
```

Przypisania bitowe: ^=, |=, <<=, >>= i >>>=. Przykład: x ^= y oznacza x = x ^ y.

Operatory typu i przypisania logicznego

Operator	Znaczenie
typeof	Zwraca typ wartości, np. typeof 10 daje "number".
instanceof	Sprawdza, czy obiekt jest instancją danego typu obiektu.
&&=	Przypisuje prawą wartość, gdy lewa strona jest truthy (traktowana jako prawda).
||=	Przypisuje prawą wartość, gdy lewa strona jest falsy (traktowana jako fałsz).
??=	Przypisuje prawą wartość, gdy lewa strona jest null albo undefined.

3. Instrukcje warunkowe i pętle

if / else

Instrukcja if wykonuje kod, gdy warunek jest prawdziwy. Blok else wykonuje się, gdy warunek jest fałszywy.

```
if (temperatura > 20) {
  console.log("Ciepło");
} else {
  console.log("Chłodno");
}
```

switch

Instrukcja switch wybiera blok kodu na podstawie wartości zmiennej. Każdy wariant zapisujemy po case. Instrukcja break kończy dany wariant, a default wykonuje się, gdy nie pasuje żaden case.

```
let dzien = 2;
switch (dzien) {
  case 1:
    console.log("Poniedziałek");
    break;
  case 2:
    console.log("Wtorek");
    break;
  default:
    console.log("Inny dzień");
}
```

Uwaga na break: Bez break program może przejść do kolejnego case, nawet jeśli jego wartość nie pasuje.

Pętle

Pętla powtarza instrukcje. Wybór pętli zależy przede wszystkim od tego, czy znamy liczbę powtórzeń i kiedy sprawdzamy warunek.

Pętla	Jak działa	Kiedy używać
for	Ma inicjalizację, warunek i zmianę zmiennej sterującej.	Gdy zwykle znamy liczbę powtórzeń.
while	Najpierw sprawdza warunek, a potem wykonuje instrukcje.	Gdy liczba powtórzeń zależy od warunku.
do...while	Najpierw wykonuje instrukcje, a dopiero potem sprawdza warunek.	Gdy kod ma wykonać się przynajmniej raz.

```
// FOR - od 0 do 4
for (let i = 0; i < 5; i++) {
  console.log(i);
}

// WHILE
let i = 0;
while (i < 5) {
  console.log(i);
  i++;
}

// DO-WHILE
let j = 0;
do {
  console.log(j);
  j++;
} while (j < 5);
```

Różnica do zapamiętania: while może wykonać się zero razy, jeśli warunek od początku jest fałszywy. do...while wykona się co najmniej raz.

4. Tablice

Tablica przechowuje wiele wartości pod jedną nazwą. Elementy tablicy mają indeksy, czyli numery pozycji. Indeksowanie zaczyna się od 0.

```
const liczby = [10, 20, 30];
console.log(liczby[0]); // 10
console.log(liczby[2]); // 30
console.log(liczby.length); // 3
```

- Element - pojedyncza wartość znajdująca się w tablicy.
- Indeks - numer pozycji elementu, liczony od 0.
- length - właściwość zwracająca liczbę elementów tablicy.
- Tablica może przechowywać wartości różnych typów danych.
- Tablice wielowymiarowe to tablice, których elementami są inne tablice.

Przechodzenie po tablicy

Pętlą for można przejść po indeksach tablicy. Konstrukcja for...in również przechodzi po kluczach, czyli w przypadku zwykłej tablicy najczęściej po indeksach.

```
for (let i = 0; i < liczby.length; i++) {
  console.log(liczby[i]);
}

for (const indeks in liczby) {
  console.log(liczby[indeks]);
}
```

Dodawanie i usuwanie elementów

Metoda	Działanie	Przykład
push(x)	Dodaje element na koniec tablicy.	tablica.push(5)
unshift(x, y)	Dodaje elementy na początku tablicy.	tablica.unshift(1, 2)
pop()	Usuwa i zwraca ostatni element.	tablica.pop()
shift()	Usuwa i zwraca pierwszy element.	tablica.shift()
splice(start, ile)	Usuwa wskazaną liczbę elementów od podanego indeksu.	tablica.splice(3, 2)
splice(start, 0, x, y)	Wstawia elementy w podanym miejscu bez usuwania.	tablica.splice(2, 0, 7, 8)

W splice(start, deleteCount, ...elementy): start oznacza indeks początku działania, deleteCount - liczbę usuwanych elementów, a kolejne argumenty są wartościami do wstawienia. Metoda zmienia oryginalną tablicę i zwraca usunięte elementy.

Sumowanie elementów tablicy

Sumę można obliczyć na kilka sposobów. Najłatwiej zacząć od zwykłej pętli for:

```
let suma = 0;
for (let i = 0; i < liczby.length; i++) {
  suma += liczby[i];
}
```

forEach wykonuje podaną funkcję dla każdego elementu tablicy. W funkcji można otrzymać element oraz jego indeks.

```
let suma = 0;
liczby.forEach((element, indeks) => {
  suma += element;
});
```

reduce przetwarza elementy i sprowadza je do jednej wartości, np. sumy. Pierwszy argument funkcji to dotychczasowy wynik, a drugi - aktualny element.

```
const suma = liczby.reduce((total, element) => {
  return total + element;
}, 0);
```

Filtrowanie tablicy

filter tworzy nową tablicę z tych elementów, dla których warunek zwrócił true. Nie zmienia oryginalnej tablicy.

```
// Tylko elementy będące tablicami
const tablice = dane.filter(el => Array.isArray(el));

// Tylko teksty
const teksty = dane.filter(el => typeof el === "string");

// Tylko liczby
const liczby = dane.filter(el => typeof el === "number");
```

Ważne: Array.isArray(x) sprawdza, czy x jest tablicą. typeof x zwraca nazwę typu, np. "string" albo "number".

5. Łańcuchy znaków (String)

String to typ danych służący do przechowywania tekstu. Tekst zapisujemy w cudzysłowie, apostrofach lub znakach odwrotnego apostrofu (backtick).

Zapis	Cechy	Przykład
"..."	Zwykły zapis tekstu w podwójnym cudzysłowie.	let tekst = "Ala";
'...'	Zwykły zapis tekstu w apostrofach. W środku łatwo umieścić cudzysłów.	let tekst = 'Powiedział: "cześć"';
`...`	Pozwala wstawiać wartości zmiennych przez ${...}.	let tekst = `Cześć ${imie}`;

Zapis z ${zmienna} nazywa się interpolacją. Dzięki niemu nie trzeba łączyć tekstu operatorem +.

```
let imie = "Ala";
let tekst = `Cześć, ${imie}!`;
```

Kody znaków i String.fromCharCode()

String.fromCharCode(kod) zamienia podany kod liczbowy na znak. Można dzięki temu przechodzić po kodach znaków i budować tekst.

```
let tekst = "";
for (let kod = 33; kod <= 255; kod++) {
  tekst += String.fromCharCode(kod);
}
console.log(tekst);
```

Doprecyzowanie: W oryginalnym materiale zakres 0-255 nazwano ASCII. Ściśle mówiąc, podstawowy ASCII obejmuje kody 0-127; zakres 128-255 zależy od użytego rozszerzenia kodowania. String.fromCharCode działa na kodach jednostek UTF-16.

Losowanie znaków

Math.random() losuje liczbę z zakresu od 0 (włącznie) do 1 (bez 1). Math.trunc() usuwa część ułamkową. Wzór z materiału pozwala wylosować kod z przedziału min - max, przy czym max nie jest wliczany.

```
let dlugosc = 30;
let max = 255;
let min = 33;
let tekst = "";

for (let i = 0; i < dlugosc; i++) {
  let kod = Math.trunc(Math.random() * (max - min)) + min;
  tekst += String.fromCharCode(kod);
}
console.log(tekst);
```

Sortowanie znaków

Metoda sort() sortuje elementy tablicy, a nie zwykły tekst typu String. Dla kodów liczbowych warto podać funkcję porównującą, aby sortowanie było rosnące liczbowo.

```
const kody = [67, 65, 66];
kody.sort((a, b) => a - b);
const tekst = String.fromCharCode(...kody); // "ABC"
```

Zapis ...kody rozpakowuje elementy tablicy jako osobne argumenty funkcji String.fromCharCode().

Znaki specjalne i encje HTML

-   to encja HTML oznaczająca twardą spację, czyli odstęp, który nie powinien zostać przełamany na końcu wiersza.
- ♠, ♥, ♦ i ♣ to encje HTML oznaczające symbole kart. Są interpretowane przez HTML, a nie przez zwykły tekst w konsoli.

6. Funkcje

Funkcja to wydzielony fragment kodu, który można uruchamiać wtedy, gdy jest potrzebny. Może przyjmować argumenty (dane wejściowe) i zwracać wynik za pomocą return.

Funkcja bezargumentowa

```
function przywitaj() {
  console.log("Witaj!");
}

przywitaj();
```

Funkcja bezargumentowa nie otrzymuje danych w nawiasach. Wywołujemy ją, podając nazwę i nawiasy.

Funkcja z argumentami

```
function dodaj(a, b) {
  console.log(a + b);
}

dodaj(3, 4);
```

a i b to parametry funkcji, a wartości 3 i 4 przekazane podczas wywołania to argumenty.

Funkcja zwracająca wartość

```
function dodaj(a, b) {
  return a + b;
}

let wynik = dodaj(3, 4); // wynik = 7
```

return kończy wykonywanie funkcji i przekazuje wartość z powrotem do miejsca wywołania. Funkcja może następnie zostać użyta np. po prawej stronie przypisania.

7. Wyrażenia regularne (RegExp)

Wyrażenie regularne to wzorzec opisujący tekst, którego szukamy lub który chcemy sprawdzić. Pozwala opisać wiele możliwych zapisów za pomocą krótkiej reguły.

W JavaScript często zapisuje się je między ukośnikami, np. /^\d{2}-\d{3}$/. Znaki specjalne mają określone znaczenie i nie zawsze oznaczają same siebie.

Znaki i grupy dopasowania

Zapis	Znaczenie
[abc]	Dowolny jeden znak z podanego zbioru: a, b lub c.
[^abc]	Dowolny jeden znak spoza podanego zbioru.
[a-z0-9]	Dowolna mała litera od a do z albo cyfra od 0 do 9.
\w	Znak słowa: litera A-Z lub a-z, cyfra albo _.
\W	Znak, który nie należy do grupy \w.
\s	Biały znak, np. spacja, tabulator lub znak nowej linii.
\S	Znak, który nie jest białym znakiem.
\d	Dowolna cyfra.
\D	Dowolny znak, który nie jest cyfrą.
.	Dowolny znak z wyjątkiem terminatorów linii w domyślnym trybie.

Położenie dopasowania

Zapis	Znaczenie
^	Początek tekstu (lub wiersza w trybie wielowierszowym).
$	Koniec tekstu (lub wiersza w trybie wielowierszowym).
\b	Granica słowa, np. między literą a spacją. Nie oznacza dowolnego miejsca w środku słowa.
\B	Miejsce, które nie jest granicą słowa.

Ile razy znak ma wystąpić?

Zapis	Znaczenie
*	Zero lub więcej razy.
+	Jeden lub więcej razy.
?	Zero lub jeden raz.
{n}	Dokładnie n razy.
{n,}	Co najmniej n razy.
{n,m}	Od n do m razy, włącznie.

Pozostałe elementy

Zapis	Znaczenie
\	Znak ucieczki - pozwala zapisać znak specjalny dosłownie albo użyć sekwencji specjalnej.
\x20	Znak o podanym kodzie szesnastkowym (tu: 20).
\u0020	Znak Unicode zapisany za pomocą czterech cyfr szesnastkowych.
(...)	Grupowanie części wzorca.
|	Alternatywa - pasuje lewa albo prawa część wzorca.

Przykłady wzorców z materiału

Co sprawdza?	Wzorzec	Wyjaśnienie
Kod pocztowy	^\d{2}-\d{3}$	Dwie cyfry, myślnik i trzy cyfry. Cały tekst ma pasować do wzorca.
Prosty zapis adresu WWW	^www\..+\..+$	Tekst zaczyna się od www., zawiera dalszy ciąg, kropkę i kolejną część.
Liczba całkowita lub ułamkowa	^-?\d+(\.\d+)?$	Opcjonalny minus, co najmniej jedna cyfra i opcjonalnie kropka oraz część ułamkowa.

Warto rozumieć wzorzec: W ^\d{2}-\d{3}$ znaki ^ i $ pilnują, aby dopasowany był cały tekst, \d oznacza cyfrę, a {2} i {3} określają ich liczbę.

Przykłady w notatkach są prostymi wzorcami do nauki składni. Szczególnie wzorzec adresu WWW nie sprawdza wszystkich możliwych poprawnych adresów internetowych.

8. Najważniejsze rzeczy do zapamiętania

- Indeksy tablic zaczynają się od 0, a length podaje liczbę elementów.
- push() i unshift() dodają elementy; pop() i shift() usuwają elementy.
- splice() może jednocześnie usuwać i wstawiać elementy w środku tablicy.
- forEach() wykonuje funkcję dla każdego elementu, reduce() sprowadza tablicę do jednej wartości, a filter() tworzy tablicę elementów spełniających warunek.
- if/else wybiera jedną z dróg, switch wybiera wariant, a pętle for, while i do...while powtarzają kod.
- prompt() pobiera tekst, confirm() zwraca true/false, a alert() wyświetla komunikat.
- == porównuje z możliwością konwersji typów, a === porównuje ściśle.
- ++x i x++ zwiększają zmienną o 1, ale różnią się momentem zwrócenia wartości.
- Funkcja może przyjmować argumenty i zwracać wynik przez return.
- W wyrażeniach regularnych: \d to cyfra, \s to biały znak, ^ oznacza początek, $ koniec, a {n} określa liczbę powtórzeń.

Koniec notatki: Materiał opracowano na podstawie przesłanego PDF-u. Zapisano go w bardziej uporządkowanej formie, poprawiono oczywiste błędy i doprecyzowano miejsca, które w oryginale mogły prowadzić do nieporozumień.